

1 Background

1.1 Dimensions

The constants in Table 1 define the sizes of the various arrays described later. These are from the `Config struct`.

Math	Code	Explanation
n_L	<code>n_layers</code>	number of layers in the network
n_P	<code>XX</code>	
n_S	<code>seq_len</code>	max sequence length
n_D	<code>dim</code>	transformer dimension
<code>XXXX</code>	<code>hidden_dim</code>	for ffn layers
n_V	<code>vocab_size</code>	vocabulary size
n_H	<code>n_heads</code>	number of query heads
n_{HKV}	<code>n_kv_heads</code>	number of key/value heads
n_{H_0}	<code>n_heads * head_size</code>	
n_{H_1}	<code>n_kv_heads * head_size</code>	

Table 1: Dimensions

1.2 Definitions

Definition 1 A 1D matrix is defined using a type `T` and positive integer n_X , representing the dimensions/shape. Let `M` be a 1D matrix. Then,

- $M[x]$ is a scalar of type `T`

`M` is stored as a linear array at address m_M such that

- Address of $M[x] = m_M + x$

Definition 2 A 2D matrix is defined using a type `T` and positive integers n_X, n_Y , representing the dimensions/shape. Let `M` be a 2D matrix. Then,

- $M[x]$ is a vector of length n_Y
- $M[x, y]$ is a scalar of type `T`

`M` is stored as a linear array at address m_M such that

Name	Shape	Comment	n_X	n_Y	n_Z
x	1D array	activation at current timestamp	n_D		
xb	1D array	like x but inside residual branch	n_D		
xb2	1D array	like xb	n_D		
KC	3D array	keys of kv-cache	n_L	n_P	n_S
KV	3D array	values of kv-cache	n_L	n_P	n_S

Table 2: Data structures for state

- Address of $\mathbf{M}[x] = m_{\mathbf{M}} + (x \times n_Y)$
- Address of $\mathbf{M}[x, y] = m_{\mathbf{M}} + (x \times n_Y) + y$

Definition 3 A 3D matrix is defined using a type $\mathbf{T}$ and positive integers n_X, n_Y, n_Z , representing the dimensions/shape. Let $\mathbf{M}$ be a 3D matrix. Then,

- $\mathbf{M}[x]$ is a 2D matrix of dimensions n_X, n_Y
- $\mathbf{M}[x, y]$ is a vector of length n_S
- $\mathbf{M}[x, y, z]$ is a scalar of type $\mathbf{T}$

$\mathbf{M}$ is stored as a linear array at address $m_{\mathbf{M}}$ such that

- Address of $\mathbf{M}[x] = m_{\mathbf{M}} + x \times (n_Y \times n_Z)$
- Address of $\mathbf{M}[x, y] = m_{\mathbf{M}} + (x \times n_Y \times n_Z) + (y \times n_Z)$
- Address of $\mathbf{M}[x, y, z] = m_{\mathbf{M}} + (x \times n_X \times n_Y) + (y \times n_Z) + z$

2 State

The state is represented by the data structures listed in Table 2

3 Weights

The weights are represented by the data structures listed in Table 3

Name	Shape	Comment	n_X	n_Y	n_Z
tet	2D array	Lookup table maps token IDs to vectors	n_V	n_D	
wk	3D array	key projections	n_L	n_D	n_{H_1}
wv	3D array	value projections	n_L	n_D	n_{H_1}

Table 3: Data structures for weights

Figure 1: Pseudo-code for `rmsnorm()` function

4 Code

Given

1. an integer p where $0 \leq p < n_P$
2. an integer t where $0 \leq t < n_V$

5 Instructions to LLM

For each of the functions listed in Table 4

1. Write an optimized C function
2. Return a `.c` file — the definition
3. Return a `.h` file — the declaration

Figure 2: Pseudo-code for `vecmatmul()` function

```

function foo (integer  $t$ , integer  $p$ )
  foreach  $l \in \{0, 1, \dots, n_L - 1\}$  do
     $x \leftarrow \text{tet}[t]$ 
     $\text{xb} \leftarrow \text{rmsnorm}(x, \text{wrms}[l])$ 
     $\text{q}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wq}[l])$ 
     $\text{k}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wk}[l])$ 
     $\text{v}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wv}[l])$ 

  endfor
end

```

Figure 3: Pseudo-code for `forward()` function

Function	Section Implemented
<code>rmsnorm</code>	Section 1
<code>vecmatmul</code>	Section 2
<code>forward</code>	Section 3

Table 4: Listing of functions